MediCore · App móvil del paciente (.NET MAUI, Android)

Entrega 7. Pantallas MOB-01 a MOB-12 del diseño aprobado (`Referencias/medicore/diseno`), con los textos en español, sobre la **API real**. No hay respuestas simuladas en la app ni portal web del paciente.

Qué hay y dónde

Pieza	Ruta	Se compila con
Lógica de la app sin MAUI: cliente de la API, sesión, ViewModels, estados	`src/MediCore.Mobile.Core`	SDK de .NET (en `MediCore.slnx`)
App Android (vistas XAML, almacenamiento seguro, navegación)	`src/MediCore.Mobile`	workload `maui-android` + SDK de Android (**fuera** de `MediCore.slnx`)
Pruebas unitarias de la lógica (50)	`tests/MediCore.Mobile.Tests`	SDK de .NET
Pruebas del cliente contra la API real (17)	`tests/MediCore.Api.Tests/MobileApiIntegrationTests.cs`	SDK de .NET + Docker
Flujos Maestro (8)	`tests/maestro`	Maestro + dispositivo Android
Preparación de datos y ejecución de Maestro	`scripts/maestro_prepare.py`, `scripts/maestro_run.py`	Python 3

La separación `Core` / `Mobile` permite compilar y probar casi toda la lógica sin las herramientas de Android; la capa MAUI solo contiene vistas, navegación Shell y el almacenamiento seguro.

Pantallas

ID	Pantalla	Vista	ViewModel
MOB-01	Inicio de sesión	`LoginPage`	`LoginViewModel`
MOB-02	Inicio (próxima cita, accesos, cerrar sesión)	`HomePage`	`HomeViewModel`
MOB-03	Mis citas	`AppointmentsPage`	`AppointmentsViewModel`
MOB-04	Seleccionar médico (paso 1/3)	`DoctorsPage`	`BookingViewModel`
MOB-05	Fecha y horario (paso 2/3)	`SlotsPage`	`BookingViewModel`
MOB-06	Confirmar cita (paso 3/3)	`ConfirmPage`	`BookingViewModel`
MOB-07	Detalle de cita (cancelar)	`AppointmentDetailPage`	`AppointmentDetailViewModel`
MOB-08 / MOB-09	Mis resultados / detalle	`LabResultsPage`, `LabResultDetailPage`	`LabResultsViewModel`, `LabResultDetailViewModel`
MOB-10 / MOB-11	Mis recetas / detalle	`PrescriptionsPage`, `PrescriptionDetailPage`	`PrescriptionsViewModel`, `PrescriptionDetailViewModel`
MOB-12	Notificaciones (abrir lleva al resultado)	`NotificationsPage`	`NotificationsViewModel`

La barra inferior (Inicio, Citas, Resultados, Recetas) y la campana de notificaciones siguen el diseño.

Seguridad

- **Sesión en almacenamiento seguro:** `SecureSessionStore` guarda solo los tokens y el nombre con `SecureStorage` (Keystore de Android). **Nunca** la contraseña (la prueba `Login_valido_guarda_los_tokens_pero_nunca_la_contrasena` lo comprueba); el campo se vacía tras cada intento. `android:allowBackup="false"` evita copiar la sesión a copias de seguridad.
- **Autorización en la API:** la app no decide permisos. Un paciente solo recibe lo suyo (resultados, recetas, notificaciones y citas ajenas → 404); el personal que intenta entrar a la app es rechazado (la API devuelve el rol y la app descarta y revoca ese token). Lo comprueban `MobileApiIntegrationTests` contra la API real.
- **Renovación de sesión:** el acceso dura 15 minutos y el refresco es de un solo uso; el cliente renueva antes de que venza o ante un 401, con un candado para que solo una llamada renueve. Si falla, se borra la sesión local y se vuelve a MOB-01 con "Tu sesión venció. Inicia sesión de nuevo para continuar.".
- **Red:** solo HTTPS, salvo `network_security_config.xml`, que permite HTTP únicamente hacia `10.0.2.2`, `localhost` y `127.0.0.1` (desarrollo). En QA y producción la API debe estar en HTTPS.
- **Sin secretos en el repositorio:** la dirección de la API se fija al compilar (`-p:ApiBaseUrl=...`); las contraseñas de pruebas se pasan por variable de entorno.

Estados de interfaz

Cada pantalla que carga datos muestra **cargando**, **error de conexión con "Reintentar"**, **lista vacía** (textos del diseño: "Aún no tienes citas…", etc.) y, si la sesión venció, vuelve al inicio de sesión. Formularios: el inicio de sesión valida correo y contraseña antes de llamar a la API (mensajes bajo cada campo); la reserva no deja continuar sin horario ("Selecciona un horario para continuar.") y limita la fecha entre hoy y el máximo reservable.

Reserva sin duplicados ni pérdida de datos

- El botón **Confirmar** ignora pulsaciones mientras hay una solicitud en curso (`AsyncCommand`).
- Cada solicitud lleva un `Idempotency-Key` que se **reutiliza en los reintentos** (por ejemplo, tras perder la conexión): si la primera llegó y la respuesta no, la API devuelve la cita ya creada. Cambiar de horario genera otra clave.
- Si otra persona ocupa el horario antes de confirmar (409), la app muestra "Este horario ya está ocupado. Selecciona otro para continuar.", **conserva médico y fecha**, vuelve a consultar los horarios y permite elegir otro.

Compilar, instalar y ejecutar

Herramientas necesarias (estado de este equipo al 2026-10-27, tras instalarlas)

Herramienta	Estado en este equipo	Para qué
.NET SDK 10.0.401	instalado	todo
JDK 17 (Microsoft OpenJDK 17.0.21)	instalado	compilar Android y Maestro

Herramienta	Estado en este equipo	Para qué
Workload `maui-android`	**no instalado en Windows a propósito** (la app se compila en el contenedor Linux; no hace falta para probar)	compilar `src/MediCore.Mobile` de forma nativa
SDK de Android en `%LOCALAPPDATA%\Android\Sdk` (command-line tools, platform-tools y emulator, instalados de dl.google.com)	**instalado** (no había un SDK reutilizable: Visual Studio 18 Insiders no trae uno)	instalar y ejecutar
Emulador `medicore_pixel` (Pixel 6, `system-images;android-35;google_apis;x86_64`)	**instalado y en uso**	probar la app
Maestro CLI 2.12.1 en `%LOCALAPPDATA%\Maestro\maestro\bin` (release oficial de GitHub)	**instalado**	ejecutar los flujos
Aceleración del emulador	**WHPX disponible** (`emulator -accel-check`): no hizo falta habilitar nada ni reiniciar	rendimiento

Instalación autorizada por la persona responsable; se aceptaron las licencias del SDK de Android. Smart App Control sigue activo.

Pasos (cuando estén las herramientas)

```
dotnet workload install maui-android                          # como administrador
dotnet build src/MediCore.Mobile -f net10.0-android -t:InstallAndroidDependencies -
p:AndroidSdkDirectory="$env:LOCALAPPDATA\Android\Sdk" -p:AcceptAndroidSDKLicenses=True
# Emulador (una vez): crear un AVD con la imagen instalada, o conectar un teléfono con depuración USB.
dotnet build src/MediCore.Mobile -f net10.0-android -c Debug -p:ApiBaseUrl=http://10.0.2.2:5011/
adb install -r src/MediCore.Mobile/bin/Debug/net10.0-android/com.medicore.paciente-Signed.apk
```

Compilar sin instalar el SDK en el equipo (contenedor Linux, no sustituye la prueba en Android): `scripts/android-build-linux.sh` (ver más abajo).

> **Smart App Control:** sigue activado. Los ensamblados de MAUI de Microsoft están firmados; los de MediCore no, y Windows puede bloquear su carga si alguna herramienta de compilación los ejecuta (no solo los lee). Si ocurre, compila con `scripts/android-build-linux.sh`.

Conexión con los servicios en Docker

La API y las dependencias corren en Docker en tu equipo; la app corre en Android y debe llegar a la API:

1. `docker compose up -d` (PostgreSQL, Redis, RabbitMQ) y la API en Development con `scripts/dev-stack-linux.sh up` (o `dotnet run --project src/MediCore.Api --launch-profile http` donde Smart App Control lo permita). La API queda en `127.0.0.1:5011`.
2. **Emulador de Android Studio:** el equipo anfitrión se ve como `10.0.2.2`. Compila con `-p:ApiBaseUrl=http://10.0.2.2:5011/` (valor por defecto).
3. **Teléfono por USB:** `adb reverse tcp:5011 tcp:5011` y compila con `-p:ApiBaseUrl=http://localhost:5011/`.
4. **Teléfono por Wi-Fi:** publica la API en tu red con `DEV_STACK_BIND=0.0.0.0 scripts/dev-stack-linux.sh up` (solo en una red de confianza), compila con `-p:ApiBaseUrl=http://<IP-del-equipo>:5011/` y agrega esa IP a `Platforms/Android/Resources/xml/network_security_config.xml`.

5. Para las notificaciones, RabbitMQ debe estar en marcha: el publicador y el consumidor corren dentro de la API.
6. Usuarios de prueba: `ana.garcia@example.com` (paciente de desarrollo) y `e2e.paciente@example.com` (paciente aislado de las pruebas), con la contraseña de `Seed:DevPassword`.

Compilación verificada (contenedor Linux, no Android)

`scripts/android-build-linux.sh` (imagen `medicore-android-build:local`: SDK de .NET 10.0.403, JDK 17, workload `maui-android`, Microsoft.Maui.Sdk 10.0.110, Microsoft.Android.Sdk 36.1.118; SDK de Android con la plataforma 36 instalado en el volumen nuevo `medicore-android-sdk`) **compiló la app sin errores** y generó el APK de desarrollo:

Archivo	Contenido
`artifacts/com.medicore.paciente-Signed.apk` (≈15 MB, firmado con la clave de depuración)	paquete `com.medicore.paciente`, ABIs `arm64-v8a` (teléfonos) y `x86_64` (emulador), `network_security_config.xml` incluido, API en `http://10.0.2.2:5011/`
`artifacts/com.medicore.paciente.apk`	la misma compilación sin firmar

`artifacts/` no se versiona (ignorado por Git): se regenera con el script. Qué prueba esto: que el XAML (incluidos los enlaces compilados) y el código C# **compilan** contra MAUI 10 y Android 36. Qué **no** prueba: que las pantallas se vean y funcionen en Android; eso exige un emulador o dispositivo (ver "Herramientas necesarias").

El compilador encontró y se corrigieron: `DatePicker.NewDate` es `DateTime?` en MAUI 10 (CS1503), `DisplayAlert` está obsoleto (`DisplayAlertAsync`), el restore debe incluir `android-arm64;android-x64` (NETSDK1047) y el instalador del SDK de Android exige las variables `USER`/`USERNAME` en el contenedor. Queda la advertencia `MA002` del SDK de MAUI (informativa).

Compilar en un contenedor Linux (APK de desarrollo)

`scripts/android-build-linux.sh` usa la imagen `medicore-android-build:local` (`scripts/android/Dockerfile`: SDK de .NET 10, JDK 17 y el workload `maui-android`) y un volumen nuevo `medicore-android-sdk` para el SDK de Android. Deja el APK en `artifacts/` (ignorado por Git). **Compilar en Linux produce el APK, pero no prueba la app**: eso exige un emulador o dispositivo Android.

Maestro

```
export MEDICORE_PASSWORD=<Seed:DevPassword>
python scripts/maestro_run.py            # todos los flujos; --only 03 para uno
```

`maestro_run.py` verifica antes que existan Maestro, `adb`, un dispositivo y la app instalada; si falta algo, lo lista y **no escribe ninguna evidencia**. Con todo disponible escribe `documentacion/evidencias/entrega-7-maestro.md` con el resultado real de cada flujo. Detalle de los flujos en `tests/maestro/README.md`.

Adaptaciones al diseño (explicadas)

- **Tipografías:** el diseño usa Manrope y DM Sans (Google Fonts); la app usa la tipografía del sistema para no descargar ni empaquetar fuentes. Colores, formas y textos son los del diseño.
- **Íconos de la barra inferior:** solo texto ("Inicio", "Citas", "Resultados", "Recetas").
- **Fecha de la reserva:** el diseño usa `<input type="date">`; en Android es el selector de fecha nativo.

- **Sin simulaciones:** se omiten los datos y textos de demostración del prototipo ("Acceso simulado…", "Datos ficticios de demostración", credenciales precargadas).
- **Notificaciones:** hoy el sistema genera solo `LabResultReady`; otros tipos (por ejemplo "Cita confirmada" o "Nueva receta" del diseño) llegarán con los eventos pendientes y la app los muestra sin enlace hasta entonces.
- **Cerrar sesión** está en el Inicio (como el diseño).
- **Tarjetas completas tocables** (además del enlace "Ver … →") para facilitar el uso táctil y la automatización.

Probar en el emulador (pasos reproducibles)

```
$sdk = "$env:LOCALAPPDATA\Android\Sdk"
& "$sdk\cmdline-tools\latest\bin\avdmanager.bat" create avd -n medicore_pixel -k "system-images;android-35;google_apis;x86_64" -d pixel_6
Start-Process "$sdk\emulator\emulator.exe" -ArgumentList "-avd","medicore_pixel","-no-audio","-gpu","swiftshader_indirect"
scripts/dev-stack-linux.sh up                                          # API real en Docker (127.0.0.1:5011 = 10.0.2.2 desde el emulador)
scripts/android-build-linux.sh                                         # APK en artifacts/
& "$sdk\platform-tools\adb.exe" uninstall com.medicore.paciente     # ¡siempre desinstalar antes!
& "$sdk\platform-tools\adb.exe" install artifacts\com.medicore.paciente-Signed.apk
$env:MEDICORE_PASSWORD = "<Seed:DevPassword>"; $env:MAESTRO_CLI_NO_ANALYTICS = "1"
python scripts/maestro_run.py
```

> **Desinstala antes de instalar:** cada compilación en contenedor genera una clave de depuración nueva; `adb install -r` falla con `INSTALL_FAILED_UPDATE_INCOMPATIBLE` y se seguiría probando la versión anterior. Al desinstalar se pierden los datos locales de la app (la sesión). **Mejora pendiente:** conservar una clave de depuración estable fuera del repositorio (ver H7-13 en `04-registro-de-hallazgos.md`).

Repetición final (2026-11-06): 8/8 con el código final, en emulador Android 35 contra QA (`http://10.0.2.2:8080/`). Evidencia: `documentacion/evidencias/maestro-codigo-final.md`. Antes de ejecutar hay que cerrar la cortina de notificaciones del emulador: `adb shell cmd statusbar collapse`.

Defectos que solo aparecieron al ejecutar en Android (corregidos)

1. **La app se cerraba al abrir** (`No assemblies found ... Fast Deployment`): el APK Debug no incluía los ensamblados. Ahora `EmbedAssembliesIntoApk=true` y `AndroidUseSharedRuntime=false` (el APK pesa ≈84 MB).
2. **Bibliotecas nativas comprimidas** (`lib/x86_64 MUST be STORED`): se agregó `android:extractNativeLibs="true"` al manifiesto.
3. `App` recibía `AppShell` por inyección antes de cargar los recursos; ahora lo crea `CreateWindow` y el Shell usa colores literales en su raíz.